

机器学习部分

- 成本函数 $J(w, b) = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$ 回归的目标是使 J 最小, 可以画出 J 与 w, b 的图像, 在已知的训练数据上可以找到最符合的 w 与 b , 注意此时 $J(w, b)$ 是一个三维图像,

最小值在由切面形成所有圆的中心

在取不同的 (w, b) 时可能得到相同的 $J(w, b)$ 值

- 梯度下降函数 对于非碗形的函数可能有不止一个最小值
局部最小值可能有多个

结合图像有利于更好地理解.

$$w = w - \alpha \cdot \frac{d}{dw} J(w, b) \quad b = b - \alpha \frac{d}{db} J(w, b)$$

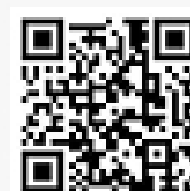
学习率 $\rightarrow$ 体现了你下坡的激进程度, 如果取的小, 则会以非常慢的速度逼近最低点

$$\frac{dJ(w, b)}{dw} = \frac{1}{n} \sum_{i=1}^m (wx_i + b - y_i) x_i$$

$$\frac{dJ(w, b)}{db} = \frac{1}{m} \sum_{i=1}^m (wx_i + b - y_i)$$

注: 成本函数的定义各教材可能有出入, 如前面系数为 $\frac{1}{2m}$ 还是 $\frac{1}{m}$ 都是可以的, 只需调整学习率即可得到完全一样的训练模型的效果

- 多维化, 有多特征时可以向量化 对应到代码实现中, 则有 `numpy.dot` $\rightarrow$ 实现两个向量相乘
- 特征缩放, 归一化, 对特征数据的处理
- 将 α 设为非常小的一个值, 如果 J 还不在下降, 则 code 有问题
在选择 α 值时可以先保守选个小的值, 然后每次乘以 3 去调试
- 特征的选择, 可以由已有的特征合成新特征, 有时更有利于成本函数拟合
- 多项式回归 都是为了更好地去拟合已有的数据



CS 扫描全能王

3亿人都在用的扫描App